



COMSATS University Islamabad, Lahore Campus
Department of Computer Engineering

Semester Project

CSC103 Programming Fundamentals

Department	Department of Computer Engineering
Section	Section B
Submitted To	Dr. Zaid Ahmad
Submission Date	23/05/2026
Group Number	Group # 09

Hotel Booking System

A Console Based Room Management Application in C++

Contents

1. Title.....	4
2. Objective.....	4
3. Problem Statement.....	4
4. Functional Modules	4
4.1 Program Flowchart.....	6
5. Concepts Used	6
6. Sample Input / Output.....	8
6.1 Displaying Room Categories	8
6.2 Booking an Available Room.....	8
6.3 Cancelling a Booking.....	9
6.4 Room Cost Estimation	10
6.5 Occupancy Summary	11
6.6 Exiting the Program	11
7. Conclusion	12
Appendix: Source Code	13

1. Title

Hotel Booking System - A Console-Based Room Management Application in C++

2. Objective

The objective of this project is to design and implement a functional hotel room booking system as a console application written in C++. The system is intended to demonstrate the practical application of core programming fundamentals, including arrays, user-defined functions, loops, conditional logic, and pointers, within the context of a real-world scenario.

The program allows users to interactively manage hotel room availability by viewing room details, making bookings, cancelling existing reservations, estimating stay costs, and reviewing hotel occupancy at any point in time. By doing so, it serves both as a working software tool and as a direct demonstration of structured programming techniques taught during the course.

3. Problem Statement

Hotels deal with a continuous flow of room requests, cancellations, and inquiries about pricing and availability. Managing this information manually or through disorganised records quickly becomes error prone and inefficient. Even at a small scale, keeping track of which rooms are occupied, which are available, and what a given stay will cost requires a structured and reliable system.

This project addresses that need by building a simplified hotel management system for a ten room hotel. The system maintains an accurate record of each room, its category, its nightly rate, and its current booking status. A guest or staff member can interact with the system through a straightforward menu to perform all essential front-desk operations. The challenge was to achieve this using only foundational C++ constructs. Without using databases or file storage, only using in-memory arrays and program logic.

4. Functional Modules

The system is divided into seven distinct functional modules. Each module handles a single, well-defined responsibility, keeping the logic clean and independently testable. The modules are described in the table below.

Module	Description
main()	The entry point of the program. It displays the main menu inside a do-while loop and dispatches control to the appropriate module based on the user's choice. It continues running until the user explicitly selects the exit option.
displayRooms()	Iterates through all ten room records using a for loop and prints each room's number, category, nightly rate, and current availability status (Booked or Available) in a formatted table.
bookRoom()	Presents the room listing to the user and accepts a room number and a duration in nights. It validates both inputs using while loops, marks the selected room as booked, and calls printBill() to display the calculated charges.
cancelBooking()	Accepts a room number from the user and verifies that the room is actually booked before proceeding. It then calls resetRoom() to release the booking, demonstrating the use of pointers.
resetRoom()	Receives a pointer to a boolean room-status variable and sets its value to false. This function exists specifically to fulfil the pointer requirement of the project while keeping the cancellation logic cleanly separated.
estimateCost()	Allows the user to calculate the expected cost of a stay for any room and any number of nights, without making a booking. It calls printBill() to display the cost breakdown.
printBill()	A helper function that computes the subtotal, applies a 16% GST charge, and prints a structured bill showing the room type, nightly rate, number of days, pre-tax total, tax amount, and final total.
occupancySummary()	Loops through the room-status array and counts how many rooms are currently booked versus available, then prints a concise summary of the hotel's current occupancy.

Table 1: Functional Modules of the Hotel Booking System

4.1 Program Flowchart

The flowchart below illustrates the overall logical flow of the program, from startup through menu-driven operation to exit. Each branch represents a separate functional module invoked by the user's menu selection.

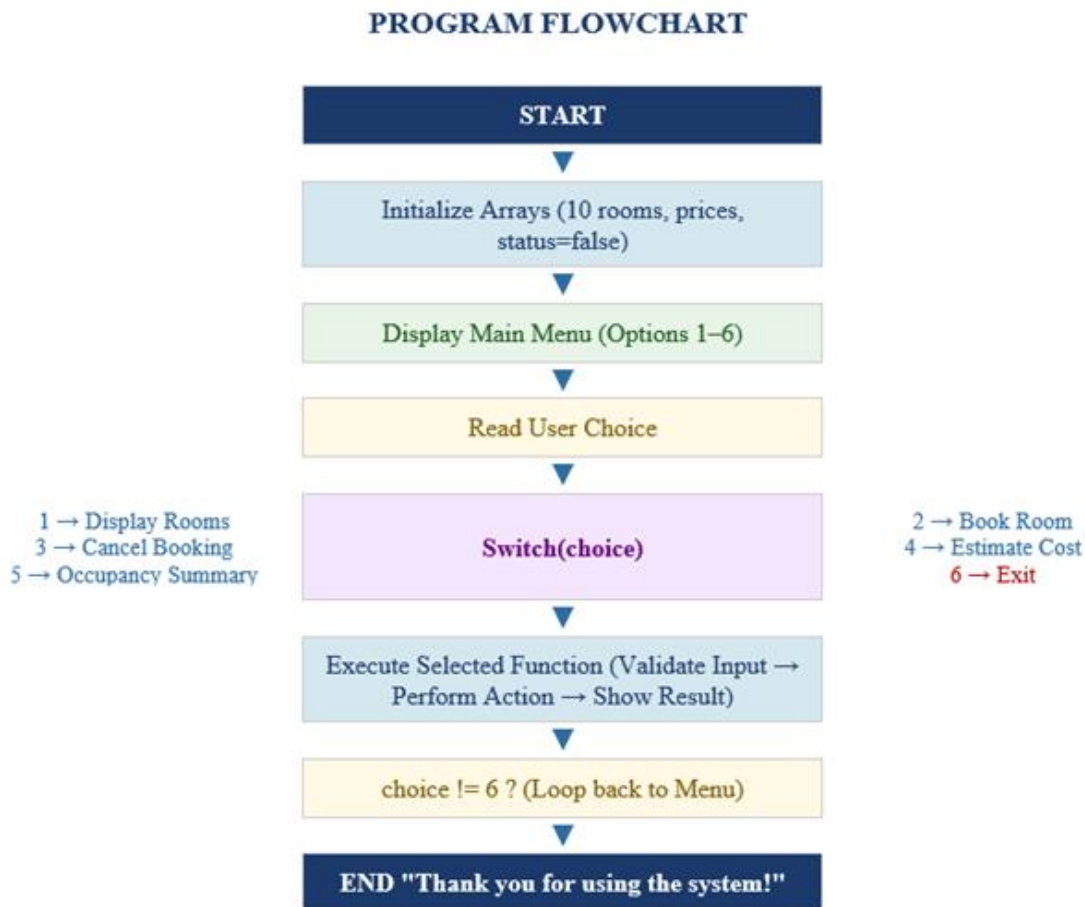


Figure 1: Program Flow Diagram — Hotel Booking System

5. Concepts Used

The program was intentionally designed to incorporate the core C++ concepts covered in the Programming Fundamentals course. The table below maps each concept to where it appears in the code and explains its role in the overall logic.

Concept	Applied In	Purpose
Arrays	roomBooked[], roomType[], roomPrice[]	Three parallel arrays store the status, category, and nightly rate for each of the ten rooms. This is the central data structure of the entire

Concept	Applied In	Purpose
		program, and all modules read from or write to these arrays.
Functions	All seven modules	The program is built entirely from user-defined functions. Each function encapsulates a single task, making the logic easier to follow and any one part easier to modify without affecting the rest.
Loops (for)	displayRooms(), occupancySummary()	A for loop in displayRooms() iterates through all rooms to print the table. Another for loop in occupancySummary() counts booked and available rooms by examining every element of the status array.
Loops (do-while, while)	main(), bookRoom(), cancelBooking(), estimateCost()	The main menu is wrapped in a do-while loop so that it always executes at least once before checking the exit condition. Input validation inside bookRoom(), cancelBooking(), and estimateCost() uses while loops to keep asking until valid input is provided.
Conditional Statements (if-else, switch)	main(), bookRoom(), cancelBooking(), displayRooms()	A switch statement in main() routes control to the correct function. if-else conditions inside bookRoom() and cancelBooking() check whether a room is available or already booked, and enforce valid numeric ranges for inputs.
Pointers	cancelBooking(), resetRoom()	The cancelBooking() function passes the address of the target element in the roomBooked array to resetRoom() using the address-of operator (&). Inside resetRoom(), the pointer is dereferenced (*) to set the value directly, demonstrating pass-by-pointer mechanics.

Table 2: C++ Concepts Applied in the Project

6. Sample Input / Output

The following screenshots were captured during a test run of the program. They illustrate the system's behaviour across all available menu options, including room display, booking, cancellation, cost estimation, occupancy reporting, and exit.

6.1 Displaying Room Categories

When the user selects option 1, the program prints a formatted table listing all ten rooms with their type, nightly rate, and current status. At startup, all rooms are available.

```
=====
HOTEL BOOKING SYSTEM
=====

1. Display Room Categories
2. Book Available Room
3. Cancel Booking
4. Estimate Room Cost
5. Display Occupancy Summary
6. Exit

Enter your choice: 1

----- ROOM DETAILS -----
Room Number   RoomType      Price/day(Rs.)   Status
-----
1             Standard      5000             Available
2             Standard      5000             Available
3             Deluxe        8000             Available
4             Deluxe        8000             Available
5             Suite         12000            Available
6             Suite         12000            Available
7             Standard      5000             Available
8             Deluxe        8000             Available
9             Suite         12000            Available
10            Standard      5000             Available
```

Figure 2: Output — Displaying All Room Categories

6.2 Booking an Available Room

The user selects option 2, views the room list, then enters a room number and a number of nights. The system validates both inputs, marks the room as booked, and prints a full bill including the base amount and 16% GST.


```

1. Display Room Categories
2. Book Available Room
3. Cancel Booking
4. Estimate Room Cost
5. Display Occupancy Summary
6. Exit

Enter your choice: 2

----- ROOM DETAILS -----
Room Number   RoomType      Price/day(Rs.)   Status
-----
1             Standard      5000             Available
2             Standard      5000             Available
3             Deluxe        8000             Available
4             Deluxe        8000             Available
5             Suite         12000            Available
6             Suite         12000            Available
7             Standard      5000             Available
8             Deluxe        8000             Available
9             Suite         12000            Available
10            Standard      5000             Available

Enter room number to book (0 to go back): 9
Enter number of days (1-30, or 0 to go back): 5

Room 9 booked successfully!

===== BILL =====
Room Type : Suite
Room Price: Rs.12000/day
Days      : 5
Bill before tax : Rs.60000
16% GST      : Rs.9600
Total Bill   : Rs.69600

```

Figure 3: Output — Booking an Available Room and Bill Generation

6.3 Cancelling a Booking

Option 3 accepts a room number from the user. It checks that the room is indeed booked before proceeding. The cancellation is performed by resetting the room's status through a pointer, and a confirmation message is displayed.

Figure 4: Output — Cancelling a Room Booking

```

1. Display Room Categories
2. Book Available Room
3. Cancel Booking
4. Estimate Room Cost
5. Display Occupancy Summary
6. Exit

Enter your choice: 3

Enter room number to cancel booking (0 to go back): 9
Booking for room 9 cancelled successfully!

```

6.4 Room Cost Estimation

Option 4 lets the user preview the cost of a stay without committing to a booking. The same bill-printing logic used during actual booking is applied here, showing how the shared printBill() helper function avoids code duplication.

```

1. Display Room Categories
2. Book Available Room
3. Cancel Booking
4. Estimate Room Cost
5. Display Occupancy Summary
6. Exit

Enter your choice: 4

----- ROOM DETAILS -----

```

Room Number	RoomType	Price/day(Rs.)	Status
1	Standard	5000	Available
2	Standard	5000	Available
3	Deluxe	8000	Available
4	Deluxe	8000	Available
5	Suite	12000	Available
6	Suite	12000	Available
7	Standard	5000	Available
8	Deluxe	8000	Available
9	Suite	12000	Available
10	Standard	5000	Available

```

[Estimate only - no booking will be made]

Enter room number to estimate (0 to go back): 10
Enter number of days (1-30, or 0 to go back): 8

===== BILL =====
Room Type : Standard
Room Price: Rs.5000/day
Days      : 8
Bill before tax : Rs.40000
16% GST       : Rs.6400
Total Bill    : Rs.46400

```

Figure 5: Output — Estimating Room Cost (No Booking Made)

6.5 Occupancy Summary

Selecting option 5 causes the program to count booked and available rooms and print a brief summary. This is useful for quickly assessing the hotel's current load without needing to read through the full room list.

```
1. Display Room Categories
2. Book Available Room
3. Cancel Booking
4. Estimate Room Cost
5. Display Occupancy Summary
6. Exit

Enter your choice: 5

===== OCCUPANCY SUMMARY =====
Total Rooms      : 10
Booked Rooms     : 1
Available Rooms  : 9
```

Figure 6: Output — Hotel Occupancy Summary

6.6 Exiting the Program

When the user enters 6, the do-while loop's exit condition is satisfied. A thank-you message is displayed and the program terminates cleanly.

```
1. Display Room Categories
2. Book Available Room
3. Cancel Booking
4. Estimate Room Cost
5. Display Occupancy Summary
6. Exit

Enter your choice: 6

Thank you for using the system!
```

Figure 7: Output — Exiting the System

7. Conclusion

The Hotel Booking System successfully demonstrates the practical application of foundational C++ programming concepts in a realistic, menu driven software application. Throughout the development process, each team member engaged with the core building blocks of structured programming: organising data in arrays, dividing behaviour into discrete functions, controlling program flow with loops and conditionals, and using pointers to manipulate memory directly.

The most significant design decision was the use of parallel arrays to represent room data. While a struct would typically be the preferred approach in larger projects, parallel arrays were chosen deliberately to stay within the scope of the course material, and they proved sufficient for a ten-room system. The consistent use of input-validation loops also ensured the program behaves predictably even when a user enters unexpected values.

The project met all stated functional goals: rooms can be displayed, booked, and cancelled; cost estimates can be generated without committing to a booking; and the hotel's overall occupancy can be reviewed at any time. All five required programming concepts: arrays, functions, loops, conditional statements, and pointers, are present and clearly exercised in the code.

Future improvements could include file-based persistence so that booking data survives program restarts, user authentication to distinguish between guest and administrator roles, and the use of structures to group room data more cleanly. Even by omitting the use of structures and file handling, the system is complete and functional.

Appendix: Source Code

The complete source code for the project is provided below for reference. The code was written in C++ and compiled using the Dev-C++ IDE.

```

/*****
* @file : main.cpp
* @project : Programming Fundamentals Lab Final Project (GROUP 9)
* @description : Hotel Booking System (C++)
* @authors : Madha Eimaan (FA25-BCE-024)
              Bilal M Naveed (FA25-BCE-014)
              Hadi Naushad (FA25-BCE-100)
* @date : 2026-19-05
*****/

// =====
// HOTEL BOOKING SYSTEM PROJECT
// =====
// Concepts Used:
// 1. Arrays
// 2. Functions
// 3. Loops
// 4. Conditional Statements
// 5. Pointers
//
// Features:
// - Display room categories
// - Book rooms
// - Cancel booking
// - Estimate room cost
// - Calculate bill
// - Display occupancy summary
// =====

#include <iostream>
#include <string>
using namespace std;

// Total rooms in hotel
const int TOTAL_ROOMS = 10;

// Arrays to store room data
bool roomBooked[TOTAL_ROOMS] = {false};

// Room categories
string roomType[TOTAL_ROOMS] =
{
    "Standard", "Standard",      // blank spaces inside strings are for formatting
    "Deluxe ", "Deluxe ",
    "Suite ", "Suite ",
    "Standard", "Deluxe ",
    "Suite ", "Standard"
};

// Room prices per night
int roomPrice[TOTAL_ROOMS] =
{
    5000, 5000,
    8000, 8000,
    12000, 12000,
    5000, 8000,
    12000, 5000
};

```

```
void displayRooms(); // Function to display all room categories
void printBill(int roomNo, int days); // Helper function to print bill breakdown
void bookRoom(); // function to book a room
void resetRoom(bool *status); // implementing pointers
void cancelBooking(); // fucntion to cancel room booking
void estimateCost();
void occupancySummary();

// =====
// Main Function
// =====
int main()
{
    int choice;
    cout << "\n\n===== \n";
    cout << "HOTEL BOOKING SYSTEM\n";
    cout << "===== \n";

    do
    {
        cout << "\n\n" << endl;
        cout << "1. Display Room Categories\n";
        cout << "2. Book Available Room\n";
        cout << "3. Cancel Booking\n";
        cout << "4. Estimate Room Cost\n";
        cout << "5. Display Occupancy Summary\n";
        cout << "6. Exit\n";

        cout << "\nEnter your choice: ";
        cin >> choice;

        switch(choice)
        {
            case 1:
                displayRooms();
                break;

            case 2:
                bookRoom();
                break;

            case 3:
                cancelBooking();
                break;

            case 4:
                estimateCost();
                break;

            case 5:
                occupancySummary();
                break;

            case 6:
                cout << "\nThank you for using the system!\n";
                break;

            default:
                cout << "\nInvalid choice! Try again.\n";
        }

        } while(choice != 6);

    return 0;
}
```

```
// -----  
// Function to display all room categories  
// -----  
void displayRooms()  
{  
    cout << "\n----- ROOM DETAILS ----- \n" << endl;  
  
    cout << "Room Number\tRoomType\t Price/day(Rs.)\t\tStatus" << endl;  
    cout << "-----" << endl;  
    for(int i = 0; i < TOTAL_ROOMS; i++)  
    {  
        cout << i + 1 << "\t\t";  
        cout << roomType[i] << "\t\t";  
        cout << roomPrice[i] << "\t\t";  
  
        if(roomBooked[i] == true)  
            cout << "Booked";  
        else  
            cout << "Available";  
  
        cout << endl;  
    }  
}  
  
// -----  
// Helper function to print bill breakdown  
// -----  
void printBill(int roomNo, int days)  
{  
    int billBeforeTax = roomPrice[roomNo] * days;  
    float gst = billBeforeTax * 0.16;  
    int totalBill = billBeforeTax + (int)gst;  
  
    cout << "\n===== BILL =====\n";  
    cout << "Room Type : " << roomType[roomNo] << endl;  
    cout << "Room Price: Rs." << roomPrice[roomNo] << "/day" << endl;  
    cout << "Days : " << days << endl;  
    cout << "Bill before tax : Rs." << billBeforeTax << endl;  
    cout << "16% GST : Rs." << gst << endl;  
    cout << "Total Bill : Rs." << totalBill << endl;  
}  
  
// -----  
// Function to book a room  
// -----  
void bookRoom()  
{  
    int roomNo, days;  
  
    displayRooms();  
  
    // --- Room number input loop ---  
    while(true)  
    {  
        cout << "\nEnter room number to book (0 to go back): ";  
        cin >> roomNo;  
  
        if(roomNo == 0)  
        {  
            cout << "Returning to main menu.\n";  
            return;  
        }  
  
        if(roomNo < 1 || roomNo > TOTAL_ROOMS)
```

```
{
    cout << "Invalid room number! Please enter a number between 1 and "
          << TOTAL_ROOMS << ".\n";
    continue;    // ask again
}

if(roomBooked[roomNo - 1] == true)
{
    cout << "Room " << roomNo << " is already booked! Please choose another room.\n";
    continue;    // ask again
}

break;    // valid and available room chosen
}

// --- Days input loop ---
while(true)
{
    cout << "Enter number of days (1-30, or 0 to go back): ";
    cin >> days;

    if(days == 0)
    {
        cout << "Returning to main menu.\n";
        return;
    }

    if(days < 1 || days > 30)
    {
        cout << "Invalid number of days! Please enter a value between 1 and 30.\n";
        continue;    // ask again
    }

    break;    // valid days entered
}

roomBooked[roomNo - 1] = true;
cout << "\nRoom " << roomNo << " booked successfully!\n";
printBill(roomNo - 1, days);
}

//-----
// function for meeting requirement of pointer implementatin
//-----
void resetRoom(bool *status)
{
    *status = false;    // * for changing the value status points to
}

// -----
// Cancel booking function
// -----
void cancelBooking()
{
    int roomNo;

    // --- Room number input loop ---
    while(true)
    {
        cout << "\nEnter room number to cancel booking (0 to go back): ";
        cin >> roomNo;

        if(roomNo == 0)
        {
            cout << "Returning to main menu.\n";
            return;
        }
    }
}
```



```
        if(roomNo < 1 || roomNo > TOTAL_ROOMS)
        {
            cout << "Invalid room number! Please enter a number between 1 and "
                << TOTAL_ROOMS << ".\n";
            continue;    // ask again
        }

        if(roomBooked[roomNo - 1] == false)
        {
            cout << "Room " << roomNo << " is not booked! Please enter a booked room number.\n";
            continue;    // ask again
        }

        break;    // valid booked room chosen
    }

    resetRoom(&roomBooked[roomNo - 1]);                // pointer implementation function

    cout << "Booking for room " << roomNo << " cancelled successfully!\n";
}

// -----
// Function to estimate room cost
// shows cost without making a booking
// -----
void estimateCost()
{
    int roomNo, days;

    displayRooms();
    cout << "\n[Estimate only - no booking will be made]\n";

    // --- Room number input loop ---
    while(true)
    {
        cout << "\nEnter room number to estimate (0 to go back): ";
        cin >> roomNo;

        if(roomNo == 0)
        {
            cout << "Returning to main menu.\n";
            return;
        }

        if(roomNo < 1 || roomNo > TOTAL_ROOMS)
        {
            cout << "Invalid room number! Please enter a number between 1 and "
                << TOTAL_ROOMS << ".\n";
            continue;    // ask again
        }

        break;    // valid room chosen (booked status doesn't matter for estimates)
    }

    // --- Days input loop ---
    while(true)
    {
        cout << "Enter number of days (1-30, or 0 to go back): ";
        cin >> days;

        if(days == 0)
        {
            cout << "Returning to main menu.\n";
            return;
        }
    }
}
```

```
        if(days < 1 || days > 30)
        {
            cout << "Invalid number of days! Please enter a value between 1 and 30.\n";
            continue;    // ask again
        }

        break;    // valid days entered
    }

    printBill(roomNo - 1, days);
}

// -----
// Function to display occupancy summary
// -----
void occupancySummary()
{
    int booked = 0;
    int available = 0;

    for(int i = 0; i < TOTAL_ROOMS; i++)
    {
        if(roomBooked[i] == true)
            booked++;
        else
            available++;
    }

    cout << "\n===== OCCUPANCY SUMMARY =====\n";
    cout << "Total Rooms      : " << TOTAL_ROOMS << endl;
    cout << "Booked Rooms      : " << booked << endl;
    cout << "Available Rooms : " << available << endl;
}
```